

Startup Ecosystem Analytics Platform

Team Members

Department of Computer Science and Engineering

srijan C Vachadmath

USN 1BM23CD060

Manohara Salmani

USN 1BM23CD035

Problem Statement I

Relational DB Bottleneck

- ▶ Multi-hop graph lookups (e.g. co-investment paths) require heavy recursive SQL JOINS.
- ▶ Query latencies degrade exponentially under deep traversals.

Unverified Data Lifecycles

- ▶ Founders self-report milestone accomplishments without validation.
- ▶ No audit trail for investor due diligence.

Concurrency & Oversubscription

- ▶ Concurrent funding calls read stale transaction state, leading to oversubscribed rounds.

Blocking API Execution

- ▶ Computational matchmaking queries block synchronous request loops.

Project Vision & Solution Overview I

Platform Objective

A high-performance startup analytics platform mapping founders, startups, and investor relationships.

- ▶ **Graph Topology:** Native property graph stores nodes and edges directly.
- ▶ **Sub-15ms Matching:** Hybrid ranking metrics calculated inside database memory.

Multi-Role Portals

- ▶ **Startups:** Manage asks, post achievements, accept interest.
- ▶ **Investors:** Set ticket bounds, check match lists, transfer funds.
- ▶ **Analysts:** Verify milestones, top up wallets, inspect network paths.

Project Vision & Solution Overview II

Unified Value Proposition

Async FastAPI API + Neo4j Graph traversals + Redis caching & mutex locks =
Scaleable, secure startup directory.

Why Neo4j? — Graph Database Choice I

Metric	Neo4j 5.13	Traditional SQL
Traversals	Index-free adjacency	Recursive JOIN tables
Edge Props	Native properties	Association entities
Flexibility	Schemaless labels	Rigid DDL migrations
Path query	Bolt protocol (<2ms)	Multiple query loops

Connection Integrity

FastAPI lifespan configures persistent connection pool to `bolt://neo4j:7687` via Neo4j python driver, using autocommit and transactional queries.

Use Case Rationale

- ▶ **Natural Graph Structure:**
Startup ecosystem is inherently relational. Connecting startups, founders, investors, and milestones via direct physical links.
- ▶ **Traversal Speed:**
Evaluates path matching queries recursively in microseconds.

Why Redis? — High-Performance Caching I

Feature	With Cache	Redis	Without Redis
Feed queries	Cached (<2ms)	hit	DB query scan (50ms)
Match lists	TTL-based cache lookup		On-the-fly math loop
Round locks	Atomic (SET NX)	lease	DB lock contention
Leaderboards	Redis sorted list	ZSET	Aggregated group-by

Caching Strategy

- ▶ **MD5 Hash Keys:** Keyed by query params, e.g. `feed:filtered:{md5}`.
- ▶ **Write Invalidation:** Mutations to profiles trigger immediate key deletes.
- ▶ **Atomic Lock Lease:** Prevents race conditions during funds transfer.

Revocation Control

JWT token IDs (JTI) blacklisted in Redis on logout for remaining token lifetime to enforce instant backend access revocation.

Neo4j Schema & Graph Design I

Neo4j Design Summary

- ▶ Native graph model stores founders, startups, investors, and achievements as connected nodes.
- ▶ Relationships such as `FOUNDED`, `INVESTED_IN`, and `HAS_ACHIEVEMENT` represent business flows.
- ▶ Cypher queries traverse multi-hop startup-investor relationships efficiently.

Entity Constraints

- ▶ Uniqueness constraint on `Startup.id`.
- ▶ Uniqueness constraint on `Investor.id`.
- ▶ Property indexes on sector and stage attributes.

Redis Architecture & Data Flow I

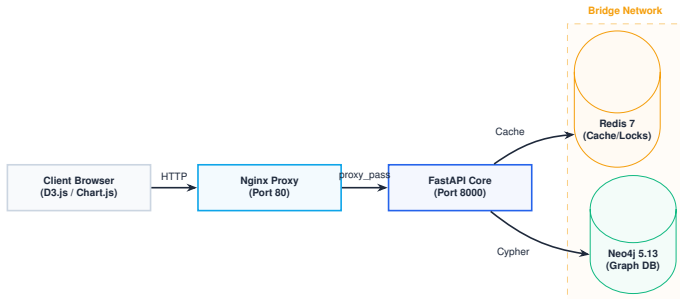
Redis Cache Flow

- ▶ FastAPI checks Redis before calling Neo4j to reduce load.
- ▶ Cache hit returns stored JSON; cache miss runs Neo4j and writes the result with TTL.
- ▶ TTL invalidation and atomic locks reduce stale reads and concurrency contention.

Storage Details

- ▶ **Hashes:**
views:startup:{id}
stores viewer metadata.
- ▶ **Sorted Sets:**
leaderboard:investors
rankings.
- ▶ **Strings:** JSON
cached payload
with explicit
expiration.

System Architecture Overview I



Gateway Security

- ▶ Nginx exposes Port 80 and reverse proxies downstream.
- ▶ Restricts DB Bolt/Redis ports to private virtual bridge.

Startup User Journey I

Startup Flow

- ▶ Register via POST /auth/signup/startup.
- ▶ Persist startup profile data and founder relationships in Neo4j.
- ▶ Post achievements with POST /achievements/post and notify interested investors.
- ▶ Accept/reject interest through POST /connect/accept.
- ▶ Receive committed funds with ZSET-backed score ordering.

Key Workflow States

- ▶ **Unverified posts:** Post achievements which go into the queue.
- ▶ **Match feed:** Retrieve matched investors based on the scoring parameters.

Investor User Journey I

Investor Flow

- ▶ Register investor profile via POST `/auth/signup/investor`.
- ▶ Browse startup feed with GET `/startups/feed`.
- ▶ Review match score through GET `/investors/{id}/matches`.
- ▶ Express interest using POST `/connect/interest`.
- ▶ Execute funding via POST `/funds/transfer`.

Financial Pipeline

- ▶ **Initial Capital:** Given \$500,000 wallet balance on signup.
- ▶ **Validation:** Asks cannot exceed available balance.

Analyst User Journey I

Analyst Flow

- ▶ Analyst logs in with role ANALYST.
- ▶ Inspects network graphs with GET `/analytics/network`.
- ▶ Verifies achievements via PATCH `/achievements/{id}/verify`.
- ▶ Tops up investor wallets with POST `/wallet/topup`.
- ▶ Audits trends through GET `/analytics/sector-trends`.

Analytics Tools

- ▶ **Global view:** Displays co-investments and connections across the entire system.
- ▶ **Authority Actions:** Verifies startup achievements to update matches.

DevOps & Infrastructure Stack I

Code & Build Automation

- ▶ **Git / GitHub:** Repository hosting source code and deployment manifests.
- ▶ **Jenkins CI/CD:** 6-stage pipeline automation defined in Jenkinsfile.
- ▶ **GitHub Actions:** Automated PR workflow validating code syntax.

Serving & Storage

- ▶ **Nginx:** Web server and reverse proxy handling static/dynamic endpoints.
- ▶ **Neo4j & Redis:** Property graph database and in-memory key-value cache.
- ▶ **Pytest Testing:** Boundary verification testing framework.

Virtualization & Runtime

Containerization Strategy I

Containerization Overview

- ▶ Nginx serves the frontend on host port 80.
- ▶ Backend runs in a private FastAPI container on port 8000.
- ▶ Redis and Neo4j remain internal to the Docker network for security.
- ▶ Docker Compose binds only the frontend public port by default.

Isolation Model

- ▶ Backend runs on a lightweight Debian-slim base image.
- ▶ Only Nginx binds to host Port 80, shielding API servers.
- ▶ Neo4j data folders are mapped to persistent Docker volumes.

Jenkins Pipeline Implementation I

Jenkins Pipeline Stages

- ▶ Checkout repository from GitHub.
- ▶ Validate Python and Docker configuration.
- ▶ Build backend and frontend containers.
- ▶ Deploy stack via Docker Compose.
- ▶ Run health checks and publish deployment results.

Safety Gates

- ▶ Automatically cleans up orphan docker networks.
- ▶ Triggers rollback if backend port 8000 health pings fail.
- ▶ Seeds empty databases on deployment verify.

Supporting Services & Automation I

Nginx Traffic Controller

- ▶ Handles static HTML file delivery directly, bypassing Python thread pools.
- ▶ Proxies dynamic `/health` and auth endpoints directly to FastAPI.
- ▶ Inject reverse proxy headers including `X-Real-IP`.

Environment Isolation

- ▶ Configuration managed through isolated `dotenv (.env)` bindings.
- ▶ Enables easy transitions between dev ports and staging settings.
- ▶ Dynamic startup hooks run database index statements automatically.

API Architecture & Route Schema I

Core FastAPI Routers

- ▶ **Startups:** /startups/feed,
/startups/register,
/startups/{id}/matches
- ▶ **Investors:**
/investors/register,
/investors/{id}/matches,
/investors/{id}/view/{sid}
- ▶ **Funds:** /funds/transfer,
/funds/history/{id},
/wallet/topup
- ▶ **Connections:**
/connect/interest,
/connect/accept

Validation Pipeline

- ▶ STRICT input checks via Pydantic schema models.
- ▶ StartupCreate & InvestorCreate reject invalid email strings or weak passwords (<6 chars) at the request layer.

Security Architecture & Access Controls I

HMAC Token Signing

- ▶ Token payloads containing subject, name, role and unique JTI token ID are signed using HMAC-SHA256.
- ▶ Expired or malformed tokens raise an immediate 401 Unauthorized code.
- ▶ Revocations validated via Redis token blacklist checking.

Role Boundary Checks

- ▶ Custom python decorators enforce role parameters per route.
- ▶ Parametrized Cypher query syntax prevents NoSQL injection attempts.
- ▶ Enforces secure password storage via salted bcrypt hashes.

Transaction Lock Lifecycle I

Lock Lifecycle

- ▶ HTTP funds transfer request acquires a Redis mutex lock.
- ▶ Neo4j transaction starts and validates balances.
- ▶ Mutations commit only after lock release and cache invalidation.

Protection Mechanics

- ▶ Mutex lock uses a 10s auto-expiry.
- ▶ Rejects transactions if funding limits are exceeded.
- ▶ Invalidates cache only after successful commit.

Pytest Testing & Verification I

Unit & Mock Framework

- ▶ Mock drivers simulate Neo4j endpoints and Redis pings in memory.
- ▶ Tests execute in isolation using GitHub Actions runner.
- ▶ Ensures database client failsafe boot modes work.

Boundary Conditions Validated

- ▶ Validates token signature tampering and expiry rejections.
- ▶ Confirms oversubscription triggers rollback.
- ▶ Validates RBAC rules for unauthorized route attempts (403 Forbidden).

Performance Optimization & Scalability I

Caching Layer Strategies

- ▶ Cache key naming incorporates MD5 parameters to avoid collision.
- ▶ Read queries bypass database for 120s feed TTL.
- ▶ Profile updates execute clear cache commands asynchronously.

Graph Performance Metrics

- ▶ In-memory index scans on ID nodes reduce lookups to microseconds.
- ▶ Co-investment paths are evaluated with Cypher paths limited to 2 hops.
- ▶ Uses Redis sorted sets for zero-latency dashboard leaderboards.

Future Roadmap & Architecture Conclusion I

Technical Growth Path

- ▶ Package stack configurations as Kubernetes Helm Charts.
- ▶ Apply Graph Data Science (GDS) algorithms for recommendations.
- ▶ Set up replica Redis caches for geographically distributed reads.

Architecture Summary

- ▶ **FastAPI Async Core:** High-concurrency throughput.
- ▶ **Neo4j Graph:** Native multi-hop relationship modeling.
- ▶ **Redis Mutex/Cache:** Prevents concurrency race conditions.